



Security Assessment Final Report



Compound Bytecode Repo

October 2025

Prepared for Compound

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Observations	4
Privileged Roles & Actors	4
Previous audits	5
Findings Summary (TODO)	6
Severity Matrix	6
Detailed Findings	7
Critical-Severely Issues	9
C-01: Anyone can send CCIP message from mainnet	9
High-Severely Issues	10
H-01: Factories can't upgrade minor and patch versions	10
Medium-Severely Issues	12
M-01: Accounting mismatch when a developer is both key and sub	12
M-02: Duplicate bytecode checks are too restrictive	14
M-03: Audited code can't be unverified	16
M-04: Cooldown is misleading	18
Low-Severely Issues	20
L-01: Gaslimit restrictions in EVM2OnRamp can cause point of failure	20
L-02: Missing storage gaps in upgradeable contracts	22
L-03: Ownership transfer doesn't clear sub developers	23
L-04: Auditor verification will fail due to incorrect hashing	25
Informational-Severely Issues	27
I-01: Unnecessary import of variables, improve readability	27
I-02: assignDeveloperForContractType ambiguity with zero address	28
I-03: NatSpec mismatch on cooldown reset authority	29
About Certora	30

Project Summary

Project Scope

Repository Name	Platform	Commit Hash
Bytecode Repo	EVM	93219299

Project Overview

This document describes the security review of Compound Bytecode Repo. The work was undertaken from **September 30th, 2025** to **October 7th, 2025**.

An additional [PR 6](#) was reviewed between **May 20th** and **May 22nd**.

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

The following files are included in our scope:

`contracts/VersionController.sol`

`contracts/L2DeployManager.sol`

`contracts/L1DeployManager.sol`

`contracts/libraries/BytecodeStore.sol`

`contracts/integration/CometConfiguration.sol`

`contracts/factories/MarketFactory.sol`

`contracts/factories/CometFactoryV2.sol`

`contracts/factories/BaseFactory.sol`

About the code

Observations

- **The cross-chain boundary must be airtight** – The CCIP receiver validation in ``L2DeployManager`` must strictly enforce both sender address and chain selector. A boolean logic error here escalates to a critical control-plane compromise (arbitrary L2 developer grants and arbitrary bytecode injection).
- **Upgrade policy needs explicit security goals**– ``CometFactoryV2`` constrains upgrades to N+1 major, which is overly restrictive and brittle for real-world governance. Conversely, there is no enforcement that upgrades use the latest verified version.
- **Cooldown semantics should protect users pre-activation, not post-release** – Cooldowns are typically used to give stakeholders the chance to react before an upgrade becomes active. Releasing and then starting the cooldown inverts the protection. Prefer staging queues and explicit activation after the cooldown passes.
- **Audit verification lifecycle lacks revocation/thresholds** – A single auditor signature marks bytecode as “verified” forever with no way to revoke or require multi-sig thresholds. Introduce revocation.
- **Re-upload policy is too strict** – A global ban on re-uploading identical init code removes legitimate rollback/determinism options and incentivizes no-op bytecode changes. Key the uniqueness on the version identifier, not on raw init code alone; or allow duplicate bytecode if version keys differ.
- **Role lifecycle on L2 is incomplete** – Developer access is time-bound but cannot be revoked early. Provide explicit revocation by timelock or by an authenticated L1 instruction.
- **Operational clarity (documentation/naming)** – Minor NatSpec inaccuracies and naming clarity issues (e.g., ``releaseBytecode`` actually performs initial release) should be corrected for maintainability and auditability. For factories, provide explicit governance upgrade entry-points to enforce verified versions.
- May 2026 PR – Allowed Guardians to now assign developers to contract types. Sending code to other chains now sends an init hash instead of the entire code itself – gas improvement. Regular Comet type was removed in favour of CometWithAssetList.

Privileged Roles & Actors

- **Governor** (`DEFAULT_ADMIN_ROLE`): global admin (Compound Governance) in `VersionController` and `L1DeployManager` upgrade authority.
- **Guardian** (`GUARDIAN_ROLE`): cooldown reset authority.
- Key Developer, Sub Developer: permitted to release bytecode and deploy using repository-managed bytecode.
- **Auditor** (`AUDITOR_ROLE`): verifies bytecode versions through EIP-712 signatures.
- **Timelock(s)**: governance executors (e.g., `CometFactoryV2.timelock`).

Previous audits

None known at the time of this review.

Findings Summary (TODO)

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	1	1	1
High	1	1	1
Medium	4	4	1
Low	4	4	3
Informational	3	3	2
Total	13	13	8

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

Issue ID	Issue Title	Severity	Status
C-01	Anyone can send CCIP message from mainnet	Critical	Fixed
H-01	Factories can't upgrade minor and patch versions	High	Fixed
M-01	Accounting mismatch when a developer is both key and sub	Medium	Fixed
M-02	Duplicate bytecode checks are too restrictive	Medium	Acknowledged
M-03	Audited code can't be unverified	Medium	Acknowledged
M-04	Cooldown is misleading	Medium	Acknowledged
L-01	Gaslimit restrictions in EVM2OnRamp can cause point of failure	Low	Fixed
L-02	Missing storage gaps in upgradeable contracts	Low	Acknowledged
L-03	Ownership transfer doesn't clear sub developers	Low	Acknowledged
L-04	Auditor verification will fail due to incorrect hashing	Low	Fixed
L-05	Developers on L2 can't be revoked immediately	Low	Fixed

I-01	Unnecessary import of variables, improve readability	Informational	Fixed
I-02	Lack of upgrade function in factories	Informational	Acknowledged
I-03	NatSpec mismatch on cooldown reset authority	Informational	Acknowledged

Critical-Severity Issues

C-01: Anyone can send CCIP message from mainnet

Severity: Critical	Impact: High	Likelihood: High
File: 134:140:contracts/L2DeployManager.sol		Status: Fixed

Description

```
134     function _ccipReceive(Client.Any2EVMMessage memory any2EvmMessage) internal override {  
135         if (  
136             abi.decode(any2EvmMessage.sender, (address)) != l1DeployManager && // @audit CRITICAL 1  
137             any2EvmMessage.sourceChainSelector != ETHEREUM_SELECTOR  
138         ) revert InvalidSender();
```

The check uses `&&` (AND). An attacker can satisfy one of the two conditions (e.g., spoof the chain selector or spoof the sender) and bypass the revert, succeeding whenever at least one side matches.

Recommendation

The correct logic is OR (`||`): if either sender or chain is wrong, revert.

Customer response

Confirmed and fixed in [commit](#).

Certora

Fix looks good.

High-Severity Issues

H-01: Factories can't upgrade minor and patch versions

Severity: High	Impact: Medium	Likelihood: High
File: 64:71:contracts/factories/CometFactoryV2.sol		Status: Fixed

Description

```
64     function setVersion(Types.VersionWithAlternative memory _newVersion) external {
65         if (msg.sender != timelock) revert OnlyTimelock();
66         if (!bytecodeProvider.versionExists(Types.BytecodeVersion(COMET_CT, _newVersion))) revert NonExistingVersion();
67         if (version.version.major + 1 != _newVersion.version.major) revert OnlyIterativeUpdate(); // @audit HIGH 1 - only
68         version = _newVersion;
69
70         emit VersionSet(_newVersion);
71     }
```

`setVersion` can't set minor and patch. Only major version increments are allowed.

Recommendation

Fix `setVersion()` to allow minor and patch upgrades.

Customer response

Confirmed. We will introduce the following rule for the version upgrade in CometFactoryV2:

- Can't set the same version.
- If the major is the same, minor should be higher than the previous minor version, patch can be any.
- If major is the same and minor is the same, patch can be any (including lowering to reflect the chosen approach of rollback within the same version).
- If a major version is to be upgraded, major should be currentMajor + 1, minor and patch can be any (as by the moment of upgrade there can be several patches or even minor version issued), ensuring the iterative update of major versions.

Thus, the major should be strictly major + 1, the minor should be greater than the previous minor. The patch can be either greater or lesser to introduce more flexibility in the upgrade process.



Certora

Agreed, correctly addressed in [commit](#).



Medium-Severity Issues

M-01: Accounting mismatch when a developer is both key and sub

Severity: Medium	Impact: Medium	Likelihood: High
File: VersionController.sol#L206-L220		Status: Fixed

Description

Current logic for fetching key developers does not consider the case where a dev is both `keyDeveloper` in one contract type and `subDeveloper` of another. This will lead to bytecode being uploaded by a party that is not the actual key developer, meddling with accounting.

Whenever bytecode is being released the author, supposedly the key developer, is fetched via `getKeyDeveloper`.

```
function releaseBytecode(
    BytecodeInput calldata _bytecodeInput
) external checkDeveloper(_bytecodeInput.contractType, msg.sender) {
    if (latestVersions[_bytecodeInput.contractType].major != 0)
        revert BytecodeAlreadyReleased(_bytecodeInput.contractType);
    VersionWithAlternative memory version = VersionWithAlternative(Version(1, 0, 0), "");
    address keyDeveloper = getKeyDeveloper(msg.sender);
    _uploadBytecode(_bytecodeInput, keyDeveloper, version);

    latestVersions[_bytecodeInput.contractType] = version.version;
    minMajorReleaseTimestamp[_bytecodeInput.contractType] = uint64(block.timestamp + MAJOR_RELEASE_COOLDOWN);
    minMinorReleaseTimestamp[_bytecodeInput.contractType][1] = uint64(block.timestamp + MINOR_RELEASE_COOLDOWN);
    minPatchReleaseTimestamp[_bytecodeInput.contractType] = uint64(block.timestamp + PATCH_RELEASE_COOLDOWN);
}
```

The key developer is passed as author of the code during uploading. The issue arises in `getKeyDeveloper`

```
function getKeyDeveloper(address _account) public view returns (address) {
    return hasRole(KEY_DEVELOPER_ROLE, _account) ? _account : subToKeyDeveloper[_account];
}
```

The function has an early return if the caller has `KEY\_DEVELOPER\_ROLE`. This is problematic since a dev can be both key and sub dev in a different contract type. If a sub developer releases the bytecode, without being a key dev elsewhere, it will return the correct dev. However, if they are key dev elsewhere, it will return early, believing they are the actual key developer, disrupting accounting.

Recommendation

Create a path considering a developer who has both roles, return the correct key dev of the contract type.

Customer response

Confirmed. The following restrictions will be added:

- accounts with the role KEY_DEVELOPER_ROLE won't be able to receive SUB_DEVELOPER_ROLE and vice versa.

This way, there shouldn't be any overlaps in the function "getKeyDeveloper()".

Certora

Fix looks good, addressed in [commit](#)

M-02: Duplicate bytecode checks are too restrictive

Severity: Medium	Impact: Medium	Likelihood: High
File: 534:539:contracts/VersionC ontroller.sol		Status: Acknowledged

Description

```

528     function _uploadBytecode(
529         BytecodeInput calldata _bytecodeInput,
530         address _keyDeveloper,
531         VersionWithAlternative memory _version
532     ) internal checkURL(_bytecodeInput.sourceURL) {
533         address[] memory initCodePointers = BytecodeStore._writeInitCode(_bytecodeInput.initCode);
534         bytes32 bytecodeHash = keccak256(_bytecodeInput.initCode);
535
536         if (isBytecodeUploaded[bytecodeHash]) revert BytecodeAlreadyUploaded(bytecodeHash); // @audit MEDIUM
537
538         isBytecodeUploaded[bytecodeHash] = true;
539     }

```

The system prevents the same initialisation code from being stored again, even if the target version key differs. This prevents rollbacks if the pushed bytecode is incorrect and the DAO wants to revert to the exact previous bytecode.

Recommendation

Scope the ban to (contractType, versionHash, initcode) rather than only raw init code hash or build a rollback mechanism.

Customer response

Acknowledged. The system acts as expected.

By design, we expect the rollback to be possible within the same minor version – if a certain patch is not agreed upon in the community, we need to have a rollback to the previous patch (e.g. 1.0.12 did not meet support, so we revert to 1.0.11). This will be ensured after the update reflected in the comments to H-02.

However, in case a revert is required to the previous minor version, we expect issuing the new version with certain changes included. By the idea, it is a place where we should deviate from the

regular practice adopted in software development, as a smart contract onchain “rollback” is a more nuanced process than the rollback of the package binary or code commit in github, (as it requires an upgrade anyway). Thus, while we expect patches to have easy “rollbacks” (as we expect patches to have NO changes into the storage), minor version are expected to have storage changes and so any minor version “rollback” should be treated as an upgrade with potentially additional changes compared to the version to “rollback”.

Additionally, same bytecodes shouldn’t be uploaded to prevent spamming with the same versions.

In case, there has to be a revocation of a whole version, there is still an option to use the previous version in the deployment not via the Factory but directly from the VersionController (since it will need to pass through the Governance anyway).

Note: we see this not as a security issue, but rather as a nuance of the deliberately chosen business logic.

Certora

Agree, the system acts as normal, we aimed to raise this concern in case it actually became a headache in the future. The issue remains as acknowledged, no need for fix.

M-03: Audited code can't be unverified

Severity: Medium	Impact: Medium	Likelihood: High
File: 343:368:contracts/VersionController.sol		Status: Acknowledged

Description

A single signature permanently sets verified to true; there is no revocation. However, a revocation mechanism could be useful if a mistake is made or a bug is found in the code afterwards.

Recommendation

Add a `revokeAudit()` function in VersionController.

Customer response

The verification of the bytecode is the authorized action which is based on the communication of developers with auditors, the audit itself, and certain security guarantees for the code – as it can be set as secured only when auditors are positive on that. Thus, the revocation of the authorized action removes the sense in the authorization, and removes the security confidence from the uploaded bytecode.

By design, the verification for the bytecode is applied after the consensus of developers and auditors, thus creating a security confidence. Additionally, in a rare case when issue is discovered post-audit, auditors can share the feedback during the proposal voting up to the cooperation on the proposal rejection, thus forcibly rejecting the compromised verified version and forcing the release of the next one.

Thus we will keep the current implementation as is as it corresponds to the described process. Any revocations of the audit results can be reflected in the Audit Report itself and in the proposal process. If the version has to be invalidated, we will be adding a patch version for it.

Also, we will add a note in the NatSpec, that the audit report might change after the validation and reflect that the version is no longer valid.

Note: we see this not as a security issue, but rather as a nuance of the deliberately chosen business logic. Thus, we request the review of the criticality of the issue or its removal from the issues section to the e.g. observations section.

Certora

Agree, code acts as expected and this is deliberately chosen logic. The reason for the report is a mirror rule in security reviews – every action is advisable to have counter-action. We accept this as a non-issue and it will remain acknowledged without need for fix.

M-04: Cooldown is misleading

Severity: Medium	Impact: Medium	Likelihood: High
File: 201:222:contracts/VersionController.sol		Status: Acknowledged

Description

The current flow uploads and “releases” at once, then sets future cooldowns. A protective cooldown should precede activation so users can exit or governance can veto. Introduce a two-step flow: stage (queue) → activate.

Recommendation

The cooldown mechanism is misleading. Either add a bytecode queue to allow every user to evaluate the next update, or remove this cooldown feature, as it adds limited value.

Customer response:

Current cooldown is implemented to prevent developers from spamming of new versions and to prevent human errors. Ideally, developers upload new versions after the audit is conducted. The current system doesn’t bypass the Timelock functionality, thus any changes to the protocol will still pass a certain period of time, allowing users to make their decision.

Therefore, cooldown serves to its goal:

- Developers engage with community and auditors on the new version to
- Developers develop new code version in cooperation with auditors
- Developer uploads new version
- Cooldown is activated for developers to prevent issuing the next version immediately
- Ideally, the verification from auditors is issued immediately after that (as it is expected that developers will have cooperation with auditors during the development, and only the audited version will be uploaded)
- Though the cooldown does not prevent issuance of the proposal, it’s review during the Timelock period, discussion, and rejection or approval from the community.

Therefore the functionality creates no security impact and does not mislead the community, since the process of Governance is not affected.

Note: we see this not as a security issue, but rather as a nuance of the deliberately chosen business logic. Thus, we request the review of the criticality of the issue or its removal from the issues section to the e.g. observations section.

Certora

We agree with the classification, the reason for reporting it was due to our experience with other similar interpretations of cooldown mechanisms and their purpose within a codebase. As clients stated, this is a non-issue and deliberate design, it is acknowledged and does not need a fix.

Low-Severity Issues

L-01: Gaslimit restrictions in EVM2OnRamp can cause point of failure

Severity: Low	Impact: Low	Likelihood: Low
File: L1DeployManager.sol		Status: Fixed

Description

Different chain configs are stored via `chainConfigs` mapping and each config has a set value for `gasLimit`. In case the gas limit is higher than the destination chain one, the CCIP send reverts.

Whenever a Chainlink CCIP is initiated, the `gasLimit` is fetched from the config and passed in `EVM2AnyMessage` as extra args.

```
function _ccipSend(uint256 _chainId, bytes memory _message) private {
    ChainConfig storage config = chainConfigs[_chainId];
    Client.EVM2AnyMessage memory evm2AnyMessage = Client.EVM2AnyMessage({
        receiver: abi.encode(config.l2DeployManager),
        data: _message,
        tokenAmounts: new Client.EVMTokenAmount[](0),
        extraArgs: Client._argsToBytes(
            Client.GenericExtraArgsV2({ gasLimit: config.gasLimit, allowOutOfOrderExecution: true })
        ),
        feeToken: address(0)
    });
    uint256 feeValue = routerClient.getFee(config.destinationChainSelector, evm2AnyMessage);
    if (feeValue > address(this).balance) revert Errors.InsufficientBalance(address(this).balance, feeValue);
    routerClient.ccipSend{ value: feeValue }(config.destinationChainSelector, evm2AnyMessage);
}
```

This gas limit is further checked within the on ramp part [here](#). The gas limit is checked against a dynamic config, which is subject to changes. In case the gas limit is lowered, any ccip calls to the destination chain will fail. Given that chain configs on `L1DeployManager` are set only by governance, we could assume that pushing a change would be rather slow, creating downtime during which no changes can be pushed to the L2s. Another potential oversight is that failed CCIP messages are retried by Chainlink which will never pass but management could rely on this backup before exploring what the cause is.

Recommendation

Perform an early check with the on ramp in the rare case the gas limit was adjusted.

Customer response

We will remove the field “gasLimit” from struct “ChainConfig”. Instead, we will add a parameter “gasLimit” to the function “sendBytecodeToOtherChain”. This way, developers will be able to pass a custom gas limit and not be blocked by the DAO to update it in config.

Certora

Agreed, fixed in [commit](#)

L-02: Missing storage gaps in upgradeable contracts

Severity: Low	Impact: Low	Likelihood: Low
File: L1DeployManager.sol		Status: Acknowledged

Description

Missing storage gaps in upgradeable contracts means either inability to add new storage variables or risking storage corruption if they are introduced forcefully.

There are 2 upgradeable contracts in the codebase – `L1DeployManager` and `VersionController` which follow the OZ Transparent Upgradeable Proxy Pattern. It is a common practice and security recommendation to add storage gaps in case new variables are to be added. Without gaps, introducing new variables is not possible without corrupting existing data.

Recommendation

Introduce storage gaps or the [OZ](#) upgradeable recommendation.

Customer response

Neither L1DeployManager nor VersionController is inherited by any smart contracts, thus we can safely add new storage variables in case of updates. In case we add inheritance of these smart contracts in the future, we will add the gap as well.

Otherwise, adding gap just introduces a meaningless increase of smart contracts size and unused storage.

Certora

Agreed, due to not being inherited right now, it does not pose a risk. In case of further improvements and added inheritance, gaps will be added. No need for fixes, issue remains acknowledged.

L-03: Ownership transfer doesn't clear sub developers

Severity: Low	Impact: Low	Likelihood: Low
File: 191:200:contracts/VersionController.sol		Status: Acknowledged

Description

```
function transferContractTypeOwnership(bytes32 _contractType, address _newKeyDeveloper) external {
    if (!hasRole(KEY_DEVELOPER_ROLE, msg.sender) || contractTypeKeyDeveloper[_contractType] != msg.sender)
        revert NotAuthorizedForContractType(_contractType, msg.sender);
    _checkRole(KEY_DEVELOPER_ROLE, _newKeyDeveloper);
    if (contractTypeKeyDeveloper[_contractType] == _newKeyDeveloper) revert SameKeyDeveloper(_newKeyDeveloper);
    contractTypeKeyDeveloper[_contractType] = _newKeyDeveloper;

    emit KeyDeveloperAssigned(_contractType, _newKeyDeveloper);
}
```

Sub-developers remain associated with the previous key developer upon transfer. This may create stale expectations or unintended permissions if roles are later reassigned. Since if the admin access is transferred back to a developer, all sub developers previously granted will see their access back, which can be misleading.

Recommendation

Clear the sub developers or add a mechanism to revoke sub developers automatically.

Customer response

Ownership transfer is not supposed to clean any roles. Key Developer along with his sub developers can still be developers over other contract types. Revoking of the roles will prevent them from maintaining new roles.

By design, the developer can "transfer" their sub-developers together with the role. Developer also has the authority to clean sub-developers before role transfer (or the "receiver" developer after the role transfer).

Note: However, we see that there are some misleading points regarding this part of the role system. We will add a role-hierarchy explanation section in the README.



Certora

Agree, design is intended. Client added clarification in the readMe to ensure this is well documented in [commit](#)

L-04: Auditor verification will fail due to incorrect hashing

Severity: Low	Impact: Low	Likelihood: Low
File: contracts/libraries/BytecodeStore.sol#L19-L34		Status: Fixed

Description

One of the contents of the `BytecodeVersion` struct is a string which is not correctly encoded according to EIP712 specification. The incorrect hash won't correspond to the one signed by the auditors, meaning the verification process will always fail.

The current design of the project aims to allow auditors to sign off-chain that a given bytecode has been audited so only verified bytecode is deployable. The core part of the verification function looks like this:

```
bytes32 bytecodeHash = computeBytecodeHash(_bytecodeVersion.contractType, _bytecodeVersion.version);
AuditStatus storage auditStatus = bytecodeAuditStatus[bytecodeHash];
bytes32 reportHash = computeAuditReportHash(bytecodeHash, bytecodes[bytecodeHash].initCodeHash, _auditReport);
address author = ECDSA.recover(_hashTypedDataV4(reportHash), _signature);

_checkRole(AUDITOR_ROLE, author);
```

The auditor signature is compared to the `reportHash` generated by `computeAuditReportHash`. However we can see inside the `\_computeBytecodeHash` that there is a string field, `version.alternative` which is not handled correctly.

```
function _computeBytecodeHash(
    bytes32 _contractType,
    IVersionController.VersionWithAlternative memory _version
) internal pure returns (bytes32) {
    return
        keccak256(
            abi.encode(
                BYTECODE_VERSION_TYPEHASH,
                _contractType,
                _version.version.major,
                _version.version.minor,
                _version.version.patch,
                _version.alternative
            )
        );
}
```

As per the [EIP712 implementation](#), strings should be encoded as the keccak256 of their content which is not performed here. This will produce a different hash than what the auditor signed, meaning the `ECDSA.recover` will either return wrong value or address(0), not passing the `_checkRole` afterwards.

Recommendation

Encode strings using `keccak256(bytes(string))`

Customer response

The result of the `"_computeBytecodeHash"` can be replicated offchain by the existing onchain method `VersionController.computeBytecodeHash()`, thus allowing replication of the same result. So, auditors have all the means to produce the correct result for the verification signature. Thus, any version of the report auditors sign can be correctly checked against the onchain record. Currently, the result of the `"_computeBytecodeHash"` is used as the key for mapping "bytecodes", the mechanism to check the existence of the version (e.g., before the report signing). Therefore, unless the correct format of the version is deliberately ignored, there is no impact on the report signature production.

Nevertheless, we want to keep this function in compliance with the EIP712 in case there are integrations that require fully offchain signing of the result of the `"_computeBytecodeHash"` or the fully offchain production of the audit report signature.

Certora

Agreed, initially submitted with higher severity due to an assumption that the signer will use an off-chain hashing which will correctly handle strings according to EIP712, therefore never producing an identical hash for the bytecode verification. We agree with the proposed severity downgrade, but still insist on following EIP specifications. Correctly performed fix in [commit](#).

Informational–Severity Issues

I-01: Unnecessary import of variables, improve readability

Severity: Informational	Impact: Low	Likelihood: Low
Files: L1DeployManager.sol, CometFactoryV2.sol, Vendor.sol		Status: Fixed

Description

Some variables and contracts are imported/declared but unused. We recommend removing them to improve readability. The mock contract `Vendor.sol` is not used anywhere in the codebase. The `AUDITOR\_ROLE` in `L1DeployManager` is also unused since auditors are checked within the version controller. `ExtConfiguration` is imported and unused in `CometFactoryV2`.

Customer response

Agreed, visibility improved [here](#)

Certora

Fix looks good.

I-02: assignDeveloperForContractType ambiguity with zero address

Severity: Informational	Impact: Low	Likelihood: Low
Files: 155:165:contracts/VersionController.sol		Status: Acknowledged

Description

```
function assignDeveloperForContractType(
    bytes32 _contractType,
    address _keyDeveloper
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (_keyDeveloper != address(0)) _grantRole(KEY_DEVELOPER_ROLE, _keyDeveloper); // @audit INFO 1 - just revert
    if (contractTypeKeyDeveloper[_contractType] == _keyDeveloper) revert SameKeyDeveloper(_keyDeveloper);
    contractTypeKeyDeveloper[_contractType] = _keyDeveloper;
    registeredContractTypes.add(_contractType);

    emit KeyDeveloperAssigned(_contractType, _keyDeveloper);
}
```

Just revert if `_keyDeveloper` is zero address.

Customer response

Acknowledged. Passing zero address is a valid case and is used to clear the ownership over the contract type. The validation is necessary so that we do not grant the KEY_DEVELOPER_ROLE to the zero address.

The usage of `address(0)` is explicitly described in the function natspec

<https://github.com/woof-software/bytecode-repository/blob/d5b614e8e2b23bba9801252fcc5eb81ea3c1e6f1/contracts/VersionController.sol#L154>

Thus, it is an expected and documented behavior.

I-03: NatSpec mismatch on cooldown reset authority

Severity: Informational	Impact: Low	Likelihood: Low
Files: VersionController.sol#L171-L183		Status: Acknowledged

Description

```

167 | // @audit INFO 2 - wrong natspec. It's the guardian who can reset the cooldown. not the governance.
168 | /// @notice Allows the Governance to reset cooldown for publishing new version.
169 | /// @param _version A type of version to reset cooldown for: major, minor or patch.
170 | /// @param _contractType A type of contract for which to reset cooldown.
171 | /// @param _major An optional parameter required for restoring determine for which major version to reset
172 | function resetCooldown(
173 |     VersionType _version,
174 |     bytes32 _contractType,
175 |     uint64 _major
176 | ) external onlyRole(GUARDIAN_ROLE) {

```

NatSpec states governance, but the function calls `onlyRole(GUARDIAN\_ROLE)`. Correct the documentation to match the code.

Customer response

Agreed, natspec improved [here](#)

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.